

Demo: the merge gate

Five short scenes show the merge gate at work on real pull requests on GitHub. Each scene is one **demo pull request**: SecureGate builds it, pushes it and opens it for you, with fake values made at random for that pull request. ACME Pay, the payment provider in the demos, does not exist, so its tokens unlock nothing.

Everything can be done from SecureGate's menu, without typing commands: double-click `Start SecureGate.cmd`, then choose **9**, "The merge gate on GitHub". Each scene also has a `make` command, for PowerShell.

This repository is public. **Only ever use demo values here, never a real key, and never merge a demo pull request.**

What you need (once)

1. **GitHub's command line**, `gh`, signed in to an account that can push to the repository:

```
powershell winget install --id GitHub.cli -e gh auth login
```

Close PowerShell and open it again after installing, so that it finds `gh`.

2. **The ruleset** that makes the check required, so that a red check locks the merge button: "The one-time setting" in The two gates (merge-gate.pdf).
3. **A check that everything is ready**: in the menu, choose **9**, then **9**; or type `make doctor`. Every line should say PASS:

```
PASS gitleaks: 8.30.1, the version the merge gate uses PASS trufflehog: 3.97.9, the version the merge gate uses PASS semgrep: 1.179.0, the version the merge gate uses PASS bandit: 1.9.4, the version the merge gate uses PASS policy: policy.yaml loads (8 rules) PASS workflow: .github/workflows/secret-gate.yml has the job secret-gate PASS gh: logged in PASS origin: Bhavik-Kadian/SecureGate PASS workflow on GitHub: .github/workflows/secret-gate.yml is on main PASS required check: main cannot be merged until secret-gate passes
```

4. **No unsaved work in the SecureGate folder**. A demo pull request refuses to start when `git status` lists changes, so that none of them can end up on GitHub. Commit or stash them first.

A demo pull request never changes your own files: SecureGate builds it in a temporary folder, from GitHub's main, on a new branch called `demo/<scene>-<date and time>`, and deletes the folder afterwards. It only ever pushes that one branch, and never touches main.

The scenes at a glance

Scene	Menu: 9, then	Or type	The check	Time
1. A harmless change	1	<code>make demo-clean</code>	green	2 min
2. A payment key in the code	2	<code>make demo-leak</code>	red: rule 8 blocks the key	3 min
3. The key deleted in a later commit	3	<code>make demo-deleted</code>	still red	2 min

Scene	Menu: 9, then	Or type	The check	Time
4. Decoys that look like secrets	4	<code>make demo-decoys</code>	green, each one explained	2 min
5. Risky handling of secrets	5	<code>make demo-risky</code>	green, with warnings	2 min
6. The AI agents (optional)	6 , then F	<code>securegate agents</code> , then <code>securegate agent-fix --pr N</code>	advice next to the red check, then a fix pull request	3 min

Short on time? Show scene 2, then scene 4: a real leak stopped, and no false alarm blocking anyone.

How each scene runs in the menu

1. Choose the scene's number. The menu shows the command it runs, such as `> securegate demo-pr leak`, and then the new pull request's address and what it contains.
2. Open it in your browser? [Y/n]: press Enter. The pull request opens on GitHub, where the check **secret-gate** starts.
3. Wait here for the merge gate's result (about a minute)? [Y/n]: press Enter. The menu waits for the check, then says what happened:

The merge gate's check on demo/leak-20261006-093015 is red (run 18234567890). SecureGate said: BLOCKED (1 block, 0 warn, 0 ignore) Merging: locked. GitHub will not let anyone merge this pull request.

4. Open the report in the dashboard? [Y/n]: press Enter for the same report in the dashboard, downloaded from GitHub. Press Enter in the menu's window to close it.

Meanwhile, reload the pull request in the browser: the check has its cross or tick, and SecureGate's comment is there.

Choose **6** at any time to see the result of the last demo pull request again, and **7** for the report of the last pull request the gate checked, whichever it was. Once the AI agents (agents.pdf) are set up, **F** lets the fix agent open a second pull request, into the last demo's branch, that reads the key from the environment instead. With commands: `make ci-report` downloads and opens that report, and `.venv\Scripts\securegate ci-report --pr 12 --wait` waits for pull request 12.

Scene 1. A harmless change (2 minutes)

Menu: 9, then 1. Or: `make demo-clean`. The pull request adds a greeting for the checkout page, `demo-app/greeting.py`, with no secret in it.

The judges see: the check turns green after about a minute. SecureGate's comment says **SecureGate: PASSED**. The box at the bottom of the pull request allows merging.

Every pull request goes through four scanners. A normal change passes in about a minute: the gate stays out of the way of honest work.

Scene 2. A payment key in the code (3 minutes)

Menu: 9, then 2. Or: make demo-leak. The pull request writes a fake ACME Pay live token straight into `demo-app/payments.py`.

The judges see:

1. The check **secret-gate** with a red cross, and the box at the bottom saying that merging is blocked.
2. SecureGate's comment, posted by github-actions: **SecureGate: BLOCKED (exit code 1)**, `demo-app/payments.py:3` with the masked token (such as `acme****x9Qz`), **rule 8: provider-keys**, found by Gitleaks and TruffleHog, and a checklist to rotate an ACME Pay key that ends with "Deleting the line is not enough: the key stays in Git history."
3. If there is time: **Details** next to the check, then **Summary**, for the same report on the check's page; and the repository's **Security** tab, where the token is a code scanning alert.

A developer pasted a payment key into the code. Two scanners found it, and the policy blocked it with rule 8. The comment says exactly where it is, shows it masked, and lists the steps to rotate the key. And nobody, not even an admin, can merge this pull request while the check is red.

Scene 3. The key deleted in a later commit (2 minutes)

Menu: 9, then 3. Or: make demo-deleted. The first commit writes the token into `demo-app/payments.py`; the second replaces it with `os.environ["ACME_PAY_API_KEY"]`. The newest code is clean.

The judges see: the check is **still red**, with the same comment as in scene 2. On the pull request's **Commits** tab, two commits: "Demo: Charge customers with ACME Pay" and "Demo: Read the ACME Pay key from the environment".

The developer noticed and deleted the key. The code is clean now, but the check is still red, because the first commit still holds the key, and anyone who can see this pull request can open that commit. SecureGate reads every commit. The only real fix is to revoke the key at the provider.

Scene 4. Decoys that look like secrets (2 minutes)

Menu: 9, then 4. Or: make demo-decoys. The pull request adds test data in `demo-app/tests/test_payments.py`: `YOUR_API_KEY_HERE`, `changeme`, an order number (a UUID) and a fake ACME Pay **test-mode** token.

The judges see: the check turns **green**. The comment says **SecureGate: PASSED WITH WARNINGS:**

- the test-mode token is a **warning**, found by Gitleaks and TruffleHog: **rule 7: tests-fixtures-docs**, because it sits in a test folder, and the comment says why it was not blocked;
- `changeme`, found by Bandit as a password, is **ignored** as a placeholder (**rule 3: placeholders**), in the folded list at the end;
- `YOUR_API_KEY_HERE` and the order number are not reported at all.

Not everything that looks like a key is one. Placeholders are ignored, and values in tests only get a warning, with the reason. So the gate does not cry wolf, and people keep trusting it.

Scene 5. Risky handling of secrets (2 minutes)

Menu: 9, then 5. Or: `make demo-risky`. The pull request adds `demo-app/client.py`, which reads the ACME Pay token from the environment the right way, then writes it to the log, and puts a database password straight into the code.

The judges see: the check turns **green**, with two warnings: **rule 13: risky-handling**, found by Semgrep, for the token written to the log (`demo-app/client.py:14`), and **rule 10: hardcoded-passwords**, found by Bandit, for the password (`demo-app/client.py:10`). Gitleaks and TruffleHog find nothing here: there is no key format to spot. That is what the code checkers are for.

Leaks are not only keys pasted into code. Here the key comes from the environment, but the code prints it into the log, where many more people can read it. Semgrep and Bandit read the code itself and catch that. These are warnings, not blocks: the developer sees them in the pull request, with what to do.

Scene 6. The AI agents (3 minutes, optional)

This needs the AI agents (`agents.pdf`) set up (menu **A**, then **4**), and works best right after scene 2.

Menu: 9, then 6. The menu waits for the gate's result, as in scene 2. If the merge gate already asked the agents, their advice is in the report; otherwise the menu asks `Ask the AI agents about these findings?` [Y/n]: press Enter. Then open the report in the dashboard and select **AI advice** at the top.

The judges see: the triage of the token, likely real, with high confidence and why; a suggested fix, a line that reads `ACME_PAY_API_KEY` from the environment; and the incident plan: revoke the key at ACME Pay, check its logs since the commit, then replace it.

Then **F** on the same screen: the fix agent opens a second pull request into the demo's branch. Open it in the browser: its change replaces the token with `os.environ["ACME_PAY_API_KEY"]`, and its description says the key is still in the history, so it must be revoked.

Three AI agents help the reviewer: one explains each finding, one writes the fix, and one plans the response. They never see a whole key: SecureGate takes every value out before anything is sent. And they only advise: the policy decided, the check is still red, and only revoking the key makes it safe.

Clean up

After the demo, close every demo pull request and delete their branches, on GitHub and here: in the menu, choose **9**, then **8**, then **y**; or type:

```
make demo-cleanup
```

It only touches pull requests and branches whose name starts with `demo/`, never `main`, and never a pull request from someone else's copy of the repository. GitHub keeps closed pull requests and their commits. That is fine here, because every value in them is fake.

Backup plan

The demo needs the internet and GitHub. Prepare for a bad connection the day before:

1. **Open the scenes in advance.** Run scene 2 (and 4, if you show it) the day before, and keep their pull requests open in browser tabs. On the day, you can show the finished checks and comments without waiting.
2. **Take screenshots** of each pull request: the check, the comment and the merge box. Also of the dashboard on the gate's report.
3. **Record a video** of one full run, from the menu to the red check. On Windows, Win+Alt+R starts and stops a recording of the active window (Xbox Game Bar).
4. **Offline, show the same report locally:** choice 2 on the main menu (or `make scan-demo-all`) writes the comment a pull request would get, and choice 3 shows it.
5. `docs/pdf/demo-script.pdf` is this page, ready to print.

If something goes wrong

What you see	What to do
the working tree has uncommitted changes	Commit or stash your changes (<code>git stash</code>), then try again.
gh is not logged in	Run <code>gh auth login</code> , then try again.
gh was not found	Install it: <code>winget install --id GitHub.cli -e</code> . Then close PowerShell or the menu, and open it again.
origin is not a GitHub repository	Start SecureGate in its own folder, the clone of the GitHub repository.
the merge gate has not run for pull request #12 yet	GitHub has not started the check yet. Wait a few seconds and choose 6 again, or add <code>--wait</code> .
Merging: NOT locked	The ruleset is missing: "The one-time setting" in The two gates (merge-gate.pdf). <code>make doctor</code> checks it.
The check stays yellow for minutes	GitHub Actions is busy. Show your screenshots, and choose 6 later.
did not finish within 15 minutes	Open the pull request's Checks tab to see what happened; GitHub may be having trouble.